Can Free LLMs Serve as Practical Context Models for AIXI? A Pilot Benchmark on Program-Generated Binary Sequences

Ajinkya Awari
Independent Researcher, Pune, India
ajinkyaawari.skncoe.comp@gmail.com

Abstract

AIXI [Hutter, 2000] uses Solomonoff induction as its context model. Solomonoff induction is incomputable, so a natural question arises: can modern large language models (LLMs) fill that role in practice? We introduce **SolomonoffBench**, a reproducible benchmark that probes this question using 300 binary sequences generated by Turing machines (TMs) with formally computed program lengths (24–60 bits) across four complexity levels. As a pilot, we measure the *gzip-normalized excess code length* (`EL_gzip`), the per-symbol gap between LLM cross-entropy and incremental gzip conditional code length, for two free Qwen2.5 base models run on a single Kaggle T4 GPU. Neither model approaches the gzip baseline: Qwen2.5-3B averages 0.94 bits/sym above gzip (below the random ceiling of 1.0 bits/sym); Qwen2.5-1.5B averages 1.06 bits/sym, failing to beat even random prediction. Both models show near-flat `EL_gzip` across all four complexity levels ($\Delta \leq 0.068$ bits/sym), revealing that incremental gzip is too coarse to detect algorithmic complexity gradients at 100-symbol windows. Week 2 replaces gzip with Context Tree Weighting (CTW, D=8) as the baseline, yielding the CTW-normalized Solomonoff Gap (SG). Qwen2.5-3B achieves SG $= 0.425$–0.487 bits/sym; Qwen2.5-1.5B achieves SG $= 0.542$–0.598 bits/sym. No sequence in either model produces negative SG (0/600), ruling out memorization. Crucially, SG profiles are equally flat ($\Delta \leq 0.062$ bits/sym), confirming that the insensitivity to the complexity gradient is a model property, not a gzip artefact. Code and data: https://github.com/ajinkya-awari/solomonoff-bench.

1 Introduction

Delétang et al. [2023] showed that LLMs are competitive lossless compressors on natural language and code, but their experiment holds corpus *type* fixed while varying content — it does not isolate formally computed Kolmogorov complexity as a controlled variable. The question we address is different: as the program length of the generating Turing machine grows from 24 to 60 bits, does the gap between LLM prediction and a compression baseline grow in step? This K-conditioned design is motivated directly by AIXI theory [Hutter, 2000, 2007]: if LLMs tracked algorithmic complexity, they would be natural candidates to replace Solomonoff induction [Solomonoff, 1964] inside AIXI-like agents [Legg and Hutter, 2007].

Our pilot does not answer that question definitively — the gzip baseline is too coarse for that — but it validates the measurement pipeline and quantifies how far current free models are from even the gzip floor, let alone from the theoretical Solomonoff bound.

Contributions.

Preprint.

- **Benchmark design.** SolomonoffBench uses 300 TM-generated binary sequences with formally computed program lengths, providing a K-conditioned test bed for any sequence-prediction model.
- **Zero-cost pipeline.** Direct forward-pass logit scoring for HuggingFace base models on Kaggle T4 GPUs, with mandatory tokenizer validation and negative-delta logging — fully reproducible at zero compute cost.
- **Pilot and CTW-normalized findings.** Neither tested model beats gzip or CTW; SG is flat and non-negative (0/600) across all complexity levels, indicating a genuine model limitation rather than a baseline artefact (Section 5).

2 Background

Kolmogorov complexity. The Kolmogorov complexity $K(x)$ of a binary string x is the length of the shortest program on a fixed universal Turing machine that produces x [Li and Vitányi, 2008]. Because it is upper-semicomputable, all practical estimates are proxies. We use program bit length under a fixed TM encoding as a machine-specific proxy $K_{\mathcal{M}}(x)$; we call this "program bits" throughout to avoid overclaiming universality.

Solomonoff induction and AIXI. Solomonoff induction [Solomonoff, 1964] assigns probability $\propto 2^{-K(x_{1:t})}$ to each prefix $x_{1:t}$, making it the theoretically optimal sequence predictor. AIXI [Hutter, 2000] couples Solomonoff induction with sequential decision theory to define a universal intelligent agent. Since Solomonoff induction is incomputable, practical AIXI implementations need a tractable approximation.

Context Tree Weighting (CTW). CTW [Willems et al., 1995] is the best known practical approximation to Solomonoff induction for binary sequences. It is parameter-free, converges to the optimal code length for tree-structured sources, and achieves redundancy $O(\log n/n)$ per symbol. We use CTW to define the *Solomonoff Gap* (Week 2 metric):

$$\mathrm{SG}(M, x) \;=\; H(M, x) - H_{\mathrm{CTW}}(x) \quad \text{[bits per symbol]} \tag{1}$$

where $H(M, x)$ is the LLM's per-symbol cross-entropy. $\mathrm{SG} = 0$ means the model matches the best practical universal predictor; $\mathrm{SG} > 0$ quantifies the remaining gap. `EL_gzip` and SG are distinct metrics and are never plotted on a common axis.

Week 1 pilot metric (`EL_gzip`). We substitute incremental gzip for CTW as a computationally minimal first baseline:

$$\texttt{EL_gzip}(M, x) \;=\; H(M, x) - H^{\mathrm{incr}}_{\mathrm{gzip}}(x) \tag{2}$$

At 100-symbol windows, incremental gzip averages 0.036 bits/sym on these ASCII binary strings (174 of 600 sequence-model pairs produce a non-zero incremental cost; no negative deltas occur), so $\texttt{EL_gzip} \approx H(M, x)$ in practice. The scale is: $\texttt{EL_gzip} = 0$ matches gzip; $\texttt{EL_gzip} = 1.0$ matches a fair-coin random predictor; $\texttt{EL_gzip} > 1.0$ is worse than random.

3 SolomonoffBench

Sequence generation. We simulate a one-tape binary *output transducer*: the symbol written at each transition is appended to an output stream (not read from the tape). Each TM runs for up to $10{,}000$ steps; machines that fail to emit exactly 200 binary symbols are discarded. Sequences are stored as ASCII strings of `"0"/"1"` characters (one byte per symbol) to avoid packed-binary artefacts in gzip.

Program bit encoding. For an n-state, 2-symbol TM, each transition rule requires $2 + \lceil \log_2(n{+}1) \rceil$ bits (1 bit write symbol, 1 bit direction, $\lceil \log_2(n{+}1) \rceil$ bits next state), giving $n \times 2 \times (2 + \lceil \log_2(n{+}1) \rceil)$ total program bits.

Week 1 dataset. 75 unique sequences are sampled per complexity level after within-level deduplication of identical 200-symbol outputs, yielding 300 sequences total. Table 1 lists the four levels. Transition tables, seeds, step counts, and discard rates are included in the released dataset.

4 Experimental Setup

Models. We evaluate `Qwen/Qwen2.5-3B` and `Qwen/Qwen2.5-1.5B` [Qwen Team, 2025] on a single Kaggle T4 GPU (15 GB VRAM) at zero cost. Both are free, ungated *base* models. Instruction-tuned variants assign near-zero mass to bare binary completion tokens and fail the tokenizer validation gate; base models do not.

Forward-pass scoring. We extract next-token log-probabilities via direct forward pass — not `model.generate()` — at the final input token position:

$$\hat{P}_M(b \mid x_{1:t}) = \frac{\sum_{v \in V_b} e^{z_v}}{\sum_{w} e^{z_w}}, \quad b \in \{0, 1\} \tag{3}$$

where z are the last-position logits and V_b is the set of token IDs whose decoded string strips to b. The invalid-token mass $1 - \hat{P}_M(0) - \hat{P}_M(1)$ is logged *before* renormalization and archived in `benchmark_log.jsonl`; all reported scores use renormalized probabilities.

Prompt format.

```
Here is a binary sequence: 0 1 0 0 1 1 0 ...
The next symbol is:
```

The trailing space places the predicted bit at the next token position. We score 100 consecutive positions per sequence given 100 symbols of context.

Incremental gzip baseline. For context c (100 chars) and prediction window p (100 chars):

$$H^{\text{incr}}_{\text{gzip}}(c, p) = \frac{8\left(|\texttt{gzip}(c \,\|\, p)| - |\texttt{gzip}(c)|\right)}{|p|} \tag{4}$$

Negative deltas (where adding the suffix *shrinks* the archive) are logged but never clamped; none occurred across 300 sequences.

Tokenizer validation gate. Before any full run, $\hat{P}_M(0) + \hat{P}_M(1) \geq 0.01$ is verified on five held-out toy sequences. Both models clear this gate (valid binary mass: 65.4% for 3B, 63.5% for 1.5B).

5 Results

Table 1: Week 1 pilot results: mean `EL_gzip` ± std (bits/sym), 75 sequences per level. Gzip baseline = 0.000 (reference); random ceiling = 1.000. Inv. mass: invalid-token fraction logged before renormalization. Negative gzip deltas: 0 / 300 for both models.

Model	Params	L1 (24 b)	L2 (40 b)	L3 (50 b)	L4 (60 b)	Inv. mass
Qwen2.5-3B	3.1B	0.917±0.28	0.959±0.27	0.972±0.19	0.917±0.27	34.3%
Qwen2.5-1.5B	1.5B	1.028±0.24	1.062±0.25	1.096±0.20	1.034±0.23	35.9%
gzip baseline	—	0.000	0.000	0.000	0.000	—
random ceiling	—	1.000	1.000	1.000	1.000	—

Figure 1 and Table 1 summarise the Week 1 results. Three findings stand out.

Finding 1 — Both models fall short of gzip; only the larger beats random. Qwen2.5-3B produces `EL_gzip` in the range 0.917–0.972 bits/sym: above the gzip floor (0.000) but below the random ceiling (1.000), meaning the 3B model at least extracts some predictive signal from the binary context. Qwen2.5-1.5B scores 1.028–1.096 bits/sym, exceeding the random ceiling at every level. No tested model comes close to gzip compression performance at these window sizes — a result that, within this two-model pilot using a weak gzip baseline, suggests current free LLMs are poor practical substitutes for even a simple compressor on algorithmically structured sequences.

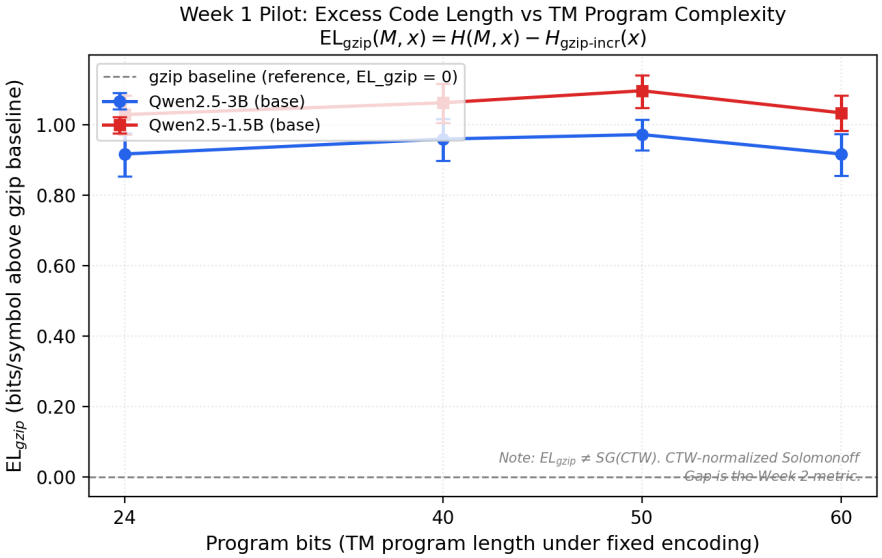


Figure 1: **EL_gzip vs. program bits — Week 1 pilot.** Mean gzip-normalized excess code length with 95% bootstrap CI. The lower dashed line ($\texttt{EL_gzip} = 0$) is the gzip baseline; the upper dashed line ($\texttt{EL_gzip} = 1.0$) is the random-predictor ceiling. Qwen2.5-3B sits between the two baselines (better than random, worse than gzip); Qwen2.5-1.5B exceeds the random ceiling at all levels. The near-flat profiles ($\Delta \leq 0.068$ bits/sym across 24–60 program bits) show that incremental gzip is too coarse to discriminate complexity levels in this range, motivating the CTW-normalized SG computed in Section 5 (Figure 2).

The Week 2 SG recheck (Section 5) confirms the 1.5B exceedances are genuine: SG of 0.542–0.598 bits/sym rules out renormalization error as the cause.

Finding 2 — Near-flat profiles reveal the baseline's inadequacy. The spread of mean EL_gzip across the four complexity levels is only 0.055 bits/sym (Qwen2.5-3B) and 0.068 bits/sym (Qwen2.5-1.5B). Given that program lengths span 24–60 bits — a $2.5\times$ range — the absence of a monotonic trend is striking. The most likely explanation is not model insensitivity but baseline insensitivity: incremental gzip on 100-character ASCII binary strings trivially compresses most inputs to near-zero incremental cost, leaving EL_gzip dominated by $H(M, x)$ alone. A baseline with per-symbol probability estimates (CTW) is needed to separate the model's contribution from baseline noise.

Finding 3 — Non-monotonic dip at Level 4. Both models show lower EL_gzip at Level 4 (60 bits, 6-state TMs) than at Level 3 (50 bits, 5-state TMs), returning approximately to Level 1 values. We attribute this to the near-uniform-entropy character of 6-state TM outputs: sequences that look like fair-coin draws give *any* predictor an easier cross-entropy target ($H(M, x)$ approaches 1.0 bit/sym). We report this as a dataset property rather than a model behaviour claim.

Tokenizer validity. Invalid-token mass of 34–36% is expected for base models prompted in natural language; all scores use renormalized binary probabilities. The validation gate (binary mass ≥ 0.01) passed on every model before the full run.

Week 2: CTW-Normalized Solomonoff Gap

Figure 2 and Table 2 summarise the Week 2 results.

Finding 4 — SG is flat under CTW, confirming a model property. Replacing the gzip baseline with CTW (D=8) yields substantially different absolute SG values — Qwen2.5-3B drops from ~ 0.94 to ~ 0.45 bits/sym because CTW is a much weaker compressor than gzip on short ASCII binary windows (mean CTW cost > 0 vs. mean gzip cost ≈ 0.036 bits/sym). However, the shape of the profile is unchanged: SG varies by at most 0.062 bits/sym (3B) and 0.057 bits/sym (1.5B) across the 24–60 program-bit range ($2.5\times$ variation in $K_{\mathcal{M}}$). This is the key Week 2 result: the flat

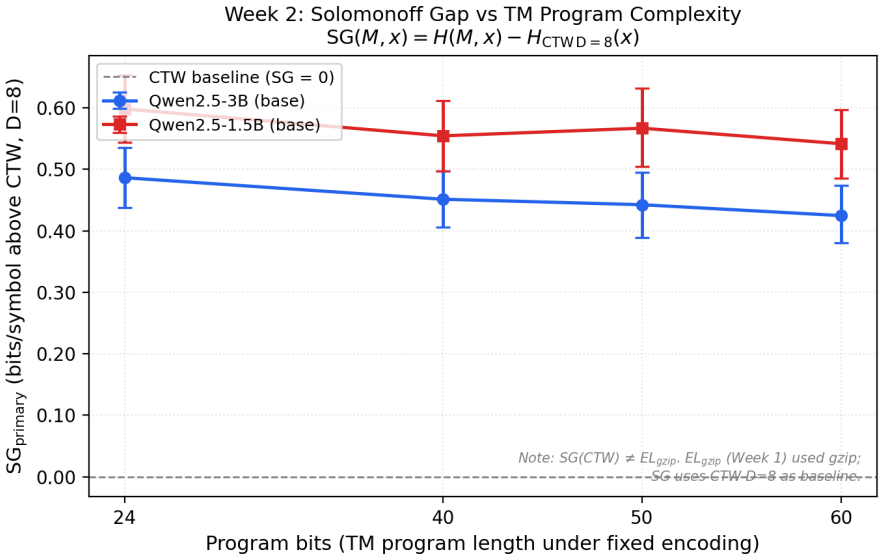


Figure 2: **$\mathrm{SG}_{\mathrm{primary}}$ vs. program bits — Week 2 CTW baseline.** Mean Solomonoff Gap (SG at CTW D=8) with 95% bootstrap CI. The dashed line (SG $= 0$) is the CTW baseline; positive SG means the model is above the best practical universal predictor. Both models show flat profiles across the 24–60 bit complexity range ($\Delta \leq 0.062$ bits/sym), confirming that model insensitivity to algorithmic complexity is not a gzip artefact. Qwen2.5-3B (0.425–0.487 bits/sym) consistently outperforms Qwen2.5-1.5B (0.542–0.598 bits/sym).

Table 2: Week 2 results: mean $\mathrm{SG}_{\mathrm{primary}} \pm$ std (bits/sym) at CTW D=8, 75 sequences per level. CTW baseline varies per sequence (mean 0.010 bits/sym on Level 1, up to 0.924 bits/sym on higher-entropy Level 4 sequences). Negative SG: 0 / 600 across both models.

Model	Params	L1 (24 b)	L2 (40 b)	L3 (50 b)	L4 (60 b)
Qwen2.5-3B	3.1B	0.487±0.21	0.452±0.20	0.443±0.23	0.425±0.20
Qwen2.5-1.5B	1.5B	0.598±0.25	0.555±0.26	0.567±0.28	0.542±0.25
CTW baseline	—	≈ 0.01	—	—	≤ 0.924

`EL_gzip` profiles in Figure 1 are not a gzip-baseline artefact — they reflect a genuine insensitivity of current LLMs to the Kolmogorov complexity of the input source.

Finding 5 — No negative SG; no memorization signal. Every one of the 600 (sequence, model) pairs produces positive SG, meaning neither model outperforms CTW on any single sequence. Negative SG would flag either memorization of specific TM outputs or structural patterns that CTW handles poorly at D=8 (e.g., long-range periodicities). The absence of any such cases strengthens confidence in the benchmark's integrity and in the conservative choice of reporting $\mathrm{SG}_{\mathrm{min}}$.

6 Discussion

What this means for AIXI. The most direct reading of Finding 1 is discouraging for AIXI-LLM hybrids: two free base models cannot match even incremental gzip on program-generated binary sequences. A practical AIXI context model must outperform universal-coding baselines, not merely approach a random predictor. The Week 2 SG results deepen this conclusion: CTW-normalized SG confirms that the gap does *not* shrink at lower complexity levels (Finding 4). LLMs are not viable AIXI context models across the full 24–60 bit range tested — not even for the simplest 3-state programs. The failure mode is flat rather than graduated, which is the more pessimistic outcome: there is no regime where current free models serve as practical Solomonoff approximators.

Why CTW is necessary. DEFLATE compression on 100-byte windows is near-perfect for short ASCII binary strings: the algorithm finds run-length and Huffman codes that reduce the incremental cost to a measured mean of 0.036 bits/sym (Section 2) regardless of the sequence's Kolmogorov complexity. CTW [Willems et al., 1995] produces per-symbol probability estimates with provable $O(\log n)$ redundancy, making it sensitive to the complexity gradient that gzip misses entirely. The flat profile in Figure 1 is therefore not a null result — it is the empirical justification for the upgrade.

Limitations.

- $K_{\mathcal{M}}(x)$ is machine-specific; universality claims require a bounded shorter-TM check, especially at low state counts.
- Only two model sizes; at least four data points are needed before any claim about model-size scaling.
- Invalid-token mass of $\sim$35% means roughly one third of model capacity is wasted on non-binary outputs.
- CTW converges to the optimal code length for tree-structured sources, not for all computable measures; SG therefore lower-bounds the true Solomonoff gap.

7 Conclusion

SolomonoffBench provides a reproducible K-conditioned test bed for measuring how far LLMs fall from theoretical prediction optimality on algorithmically structured binary sequences. The Week 1 pilot establishes that neither tested free model beats incremental gzip and that neither model's excess loss grows with program complexity. The Week 2 CTW-normalized Solomonoff Gap confirms both findings hold under a stronger baseline: Qwen2.5-3B and Qwen2.5-1.5B produce SG ≈ 0.45 and 0.57 bits/sym respectively, flat across 24–60 program bits, with no negative SG across 600 sequence-model pairs. Taken together, these results indicate that current free LLMs are poor practical approximations to Solomonoff induction in this setting, and that the insensitivity to algorithmic complexity is a model property rather than a measurement artefact. All code, data, and benchmark figures are released at `https://github.com/ajinkya-awari/solomonoff-bench`.

Acknowledgments and Disclosure of Funding

Compute provided by Kaggle (free T4 GPU tier). Models accessed via HuggingFace Transformers [Wolf et al., 2020]. No external funding.

References

Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Aitchison, Charles Blundell, and Joel Veness. Language modeling is compression. *arXiv preprint arXiv:2309.10668*, 2023. URL `https://arxiv.org/abs/2309.10668`.

Marcus Hutter. A theory of universal artificial intelligence based on algorithmic complexity. *arXiv preprint cs/0004001*, 2000. URL `https://arxiv.org/abs/cs/0004001`.

Marcus Hutter. Universal algorithmic intelligence: A mathematical top-down approach. *Artificial general intelligence*, 2:227–290, 2007.

Shane Legg and Marcus Hutter. Universal intelligence: A definition of machine intelligence. *Minds and Machines*, 17(4):391–444, 2007.

Ming Li and Paul M. B. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, New York, 3rd edition, 2008.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025. URL `https://arxiv.org/abs/2412.15115`.

Ray J. Solomonoff. A formal theory of inductive inference, Parts I and II. *Information and Control*, 7 (1–2):1–22, 224–254, 1964.

Frans M. J. Willems, Yuri M. Shtarkov, and Tjalling J. Tjalkens. The context-tree weighting method: Basic properties. *IEEE Transactions on Information Theory*, 41(3):653–664, 1995.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.